Command cheat sheet

v1.0.0-docs.1
target template v1.0.0

> Daily `gh`, task-CLI, and tracker commands. Full flags and expected output: REFERENCE · TASK CLI.

AUTH AND DISCOVERY

```
gh auth status                      # token needs 'repo' and 'project'
gh auth refresh -s project,repo     # only if a scope is missing
gh project list --owner @me --format json
gh project field-list PROJECT_NUMBER --owner @me --format json
gh project item-list PROJECT_NUMBER --owner @me --format json --limit 100
```

Option IDs, not option names, are required when editing single-select fields.

TASK CLI

```
node scripts/github/task.mjs start --issue 123 --owner OWNER [--executor agent:x]
node scripts/github/task.mjs checkpoint --issue 123 --message "Done: ... Next: ..."
node scripts/github/task.mjs review --issue 123 --pr PR_URL --evidence "tests pass"
node scripts/github/task.mjs block --issue 123 --reason "waiting" --blocked-by ISSUE_URL
node scripts/github/task.mjs done --issue 123 --evidence "npm test 11/11" [--force]
node scripts/github/task.mjs status --issue 123
```

All commands accept `--config tracker.config.json`, `--dry-run`, `--json`. Default mutates.

MANUAL FALLBACKS

```
gh project item-edit --id ITEM_ID --project-id PROJECT_ID \
  --field-id FIELD_ID --single-select-option-id OPTION_ID   # no output = ok
gh issue comment 123 --repo OWNER/REPO --body-file comment.md
gh pr create --fill --body "Closes #123"
gh pr checks 45
gh pr merge 45 --squash --delete-branch
```

TRACKER, AUDIT, AND LOCAL GATES

```
node scripts/github/validate-config.mjs tracker.config.json
npm test
npm run audit -- --config tracker.config.json --advisory   # advisory exits 0
npm run audit -- --config tracker.config.json --strict     # release gate
git status --short && git rev-parse --short HEAD
git diff --check
```

CI parity: lint/format/build commands come from `profiles.node.commands` in `tracker.config.json`; see CI profile §9.

SUB-ISSUES AND DEPENDENCIES

```
gh issue edit 123 --repo OWNER/REPO --add-blocked-by ISSUE_URL
CHILD_ID=$(gh api repos/OWNER/REPO/issues/123 --jq .id)
gh api repos/OWNER/REPO/issues/100/sub_issues -X POST -F sub_issue_id="$CHILD_ID"
```

If native sub-issues/dependencies are unavailable: use body links plus a documented fallback.

TIPS

- Discover IDs once and store them; do not paste tokens into commands or history.
- `--dry-run` first for lifecycle mutations on an unfamiliar repo.
- One task, one branch, one PR: `task/<issue>-<slug>`.
- Expected outputs and exit codes: REFERENCE §19.